Dubin's TSP in WSN

Soumili Dey

May 2026

0.1 Problem Statement

Let $MS = \{MS_1, MS_2, \ldots, MS_n\}$ denote a set of n mobile sinks (MSs) subject to non-holonomic constraints that collect data from static sensor nodes $S = \{S_1, S_2, \ldots, S_n\}$ and transport it to the base station BS in the Wireless Sensor Network (WSN). Due to their non-holonomic constraints, each MS is modeled as a Dubins vehicle and is limited to forward motion at a constant velocity v, with a minimum turning radius R. Its configuration is given by $(x, y, \theta) \in SE(2)$, where $(x, y) \in \mathbb{R}^2$ is the vehicle position in $2D$ plane and $\theta \in S^1$ is the angle at which the vehicle is headed. Its kinematics are given by:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = v \begin{bmatrix} \cos\theta \\ \sin\theta \\ \frac{\alpha}{R} \end{bmatrix}, \qquad \alpha \in [-1, 1] \tag{1}$$

where $\alpha \in [-1, 1]$ is the normalized curvature control input, with $\alpha = -1$, $\alpha = 0$, and $\alpha = 1$ corresponding to maximum right turn, straight motion, and maximum left turn, respectively. There exists a set of m static, disjoint polygonal obstacles $OB = \{ob_1, ob_2, \ldots, ob_m\}$ in the field. The objective is to minimize the time t, the distance d and the number n of MSs that collect data from all the sensor nodes, while ensuring collision-free motion with respect to both the obstacles and other MSs.

==========

0.2 System Model and Environment

Let $MS = \{MS_1, MS_2, \ldots, MS_n\}$ denote a set of n mobile sinks (MSs) that collect data from static sensor nodes $S = \{S_1, S_2, \ldots, S_n\}$ and transport it to the base station (BS).

The coordinates of sensor i and the base station are defined respectively as:

$$S_i = (x_i, y_i) \in F, \qquad i = 1, \ldots, n \tag{2}$$

$$\mathbf{S}_0 = (x_0, y_0) \in F \tag{3}$$

There exists a set of m static, disjoint polygonal obstacles $OB = \{ob_1, ob_2, \ldots, ob_m\}$. The free configuration space F is defined by removing the inflated obstacles from the $2D$ plane:

$$F = \mathbb{R}^2 \setminus \bigcup_{j=1}^{m} \mathrm{int}(O_j^{\mathrm{inflated}}) \tag{4}$$

To ensure continuous free space, obstacles are inflated by a safety clearance:

$$r_{\mathrm{safe}} = \frac{w}{2} + \delta_{\mathrm{margin}} \tag{5}$$

where w is the width of the mobile sink and δ_{margin} is an additional safety margin.

0.3 Mobile Sink Kinematics and Properties

The maximum number of heterogeneous mobile sinks available for deployment is $K_{\max}$. Each mobile sink travels at a constant velocity $v_{\max} = v$.

Due to non-holonomic constraints, each MS is modeled as a Dubins vehicle limited to forward motion. Its configuration is $(x, y, \theta) \in SE(2)$, where $(x, y) \in \mathbb{R}^2$ is the vehicle position and $\theta \in S^1$ is the heading angle. The kinematics are given by:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = v \begin{bmatrix} \cos\theta \\ \sin\theta \\ \frac{\alpha}{R} \end{bmatrix} \tag{6}$$

where $\alpha \in [-1, 1]$ is the normalized curvature control input ($\alpha = -1$ for maximum right turn, $\alpha = 0$ for straight motion, and $\alpha = 1$ for maximum left turn).

The minimum turning radius of the mobile sink is:

$$R = \frac{L}{\tan(\delta_{\max})} \tag{7}$$

where L is the vehicle's wheelbase and $\delta_{\max}$ is the maximum steering angle.

The overall length of the vehicle ℓ is:

$$\ell = \ell_{\text{rear}} + L + \ell_{\text{front}} \tag{8}$$

w is the width of the vehicle, and l is the total length of the vehicle.

0.4 Sensor Parameters

Each sensor i possesses an obstacle-aware geodesic walking distance in the free space relative to the base station:

$$r_i = d_{\text{geo}}(\mathbf{S}_0, \mathbf{S}_i) \tag{9}$$

Data harvesting at sensor i requires a stationary dwell time τ_i.

Each sensor i is also associated with a polar angle relative to the base station. Initially, this is represented as $\phi_i \in (-\pi, \pi]$:

$$\phi_i = \text{atan2}(y_i - y_0, x_i - x_0) \tag{10}$$

0.5 Decision Variables

For each potential mobile sink $k \in \{1, \ldots, K_{\max}\}$, the following variables are defined:

- **Deployment variable ($z_k \in \{0, 1\}$):** 1 if mobile sink k is deployed, 0 otherwise.
- **Sensor-assignment variable ($x_{ik} \in \{0, 1\}$):** 1 if sensor i is serviced by mobile sink k, 0 otherwise.
- **Tour sequence (Π_k):** $\Pi_k = \left(S_0, S_{\pi_{k,1}}, S_{\pi_{k,2}}, \ldots, S_{\pi_{k,n_k}}, S_0\right)$ denotes the ordered sequence of nodes visited by mobile sink k, starting and ending at the base station.
- **Heading variable ($\theta_{k,j} \in \Theta$):** The discrete arrival heading of mobile sink k at the j-th node in its tour Π_k.

0.6 Objective Functions

The problem is formulated as a lexicographic multi-objective minimization problem to minimize the number of deployed MSs (K), the maximum completion time ($T_{\max}$), and the total trajectory length (L_{total}):

$$\underset{\mathbf{z},\mathbf{x},\mathbf{\Pi},\theta}{\text{lexmin}} \, (K, T_{\max}, L_{\text{total}}) \tag{11}$$

The objectives are optimized in the following priority order:

Priority 1: Minimize the number of active mobile sinks

$$K = \sum_{k=1}^{K_{\max}} z_k \tag{12}$$

Priority 2: Minimize the parallel data-collection time

$$T_{\max} = \max_{k:z_k=1} T_k \tag{13}$$

Priority 3: Minimize the total trajectory length

$$L_{\text{total}} = \sum_{k=1}^{K_{\max}} L_k \tag{14}$$

0.7 Constraints

0.7.1 Exact Sensor Coverage

Every sensor must be assigned to exactly one deployed mobile sink:

$$\sum_{k=1}^{K_{\max}} x_{ik} = 1, \qquad \forall i \in S \tag{15}$$

A sensor can only be assigned to an MS if that MS is actually deployed:

$$x_{ik} \leq z_k, \qquad \forall i \in S, \quad \forall k \in \{1, \ldots, K_{\max}\} \tag{16}$$

0.7.2 Parallel Mission Budget

Every deployed mobile sink must complete its tour and return to the base station before the mission budget B expires. The completion time of mobile sink k combines travel time and stationary harvesting time:

$$T_k = \frac{L_k}{v} + \sum_{i \in S} x_{ik} \tau_i \tag{17}$$

$$T_k \leq B, \qquad \forall k \in \{1, \ldots, K_{\max}\} \tag{18}$$

(where L_k is the total Dubins path length along tour Π_k)

0.7.3 Obstacle Collision Avoidance

Let $\Gamma_k(t)$ denote the physical trajectory of mobile sink k. The entire trajectory must remain strictly within the free configuration space:

$$\Gamma_k(t) \in F, \qquad \forall t \in [0, T_k], \quad \forall k \in \{1, \ldots, K_{\max}\} \tag{19}$$

0.7.4 Inter-Vehicle Collision Avoidance

Any two simultaneously operating mobile sinks k and l must maintain a minimum safety distance d_{safe} between their trajectories:

$$\|\Gamma_k(t) - \Gamma_l(t)\| \geq d_{\text{safe}}, \qquad \forall k \neq l, \quad \forall t \in [0, \min(T_k, T_l)] \tag{20}$$

0.8 Proposed Radially Unimodal Heuristic

"Radially unimodal" means having exactly one peak in distance from a central starting point. Radial: Refers to the radius, which is the physical distance measured outward from the center. Unimodal: Combines "uni-" (one) and "mode" (peak or maximum). In mathematics, a unimodal sequence increases continuously, reaches a single high point, and then decreases continuously. We propose a heuristic of Genetic algorithm(GA) known as Radially Unimodal Heuristic. This heuristic hardcodes Dubins kinematics directly into the Genetic Algorithm to reduce the number of iterations required to reach the optimal fitness value. It actively prevents the GA from overexploring zig-zag paths, and converting a complex Dubins trajectory optimization problem into a simple, high-speed sorting task.1.

0.8.1 The Heuristic applied on path is as follows:

Instead of calculating the exact, complex Dubins curves for every candidate route, the GA can use Lemma 1 as a lightweight heuristic fitness function:

$$Fitness(\pi) = L_{euclid}(\pi) + m(\pi) \cdot \pi R$$

By counting the radial maxima $m(\pi)$ (how many times the sequence bounces outward and inward) and adding the Euclidean distance, the GA can rapidly rank chromosomes. It heavily penalizes solutions with high $m(\pi)$ without needing to compute the actual turning geometry.

0.8.2 The Heuristic applied to sensors is as follows:

The heuristic then takes over and strictly forces the geometry:

- Outbound Leg: Sorts assigned sensors strictly by ascending radial distance from the base station ($r \uparrow$).
- Apex:MS goes to the absolute farthest sensor.
- Inbound Leg: Sorts the remaining sensors strictly by descending radial distance ($r \downarrow$).

To mathematically formulate the Smart Population Initialization, a heuristic that forces every randomly generated starting route to have exactly one radial peak is defined. Let S_k be the subset of sensors assigned to a specific vehicle. For each sensor $i \in S_k$, its radial distance from the base station S_0 is defined as $r_i = d_{geo}(S_0, S_i)$.

Polar Angle Evaluation and Angular Sectoring (The Spatial Partition)

Before assigning routes, the global set of sensors S must be partitioned into K spatially disjoint subsets to prevent trajectory overlapping between different vehicles. For each sensor $i \in S$, its polar angle ϕ_i relative to the base station $S_0(x_0, y_0)$ is calculated as:

$$\phi_i = \text{atan2}(y_i - y_0, x_i - x_0) \in (-\pi, \pi]$$

The entire set of sensors is then sorted in ascending order of their polar angles:

$$\phi_{(1)} \leq \phi_{(2)} \leq \cdots \leq \phi_{(n)}$$

The Random Partition

Instead of randomizing the order of the stops, the algorithm only randomizes the assignment of each stop into one of two disjoint sets:

$$S_{out} \cup S_{in} = S_k \quad \text{and} \quad S_{out} \cap S_{in} = \emptyset$$

The Monotonic Sorting (The Heuristic)

Sort S_{out} in strictly ascending order of radial distance to form the outbound sequence π_{out}:

$$r_{\pi_{out,1}} \leq r_{\pi_{out,2}} \leq \cdots \leq r_{\pi_{out,|S_{out}|}}$$

Sort S_{in} in strictly descending order of radial distance to form the inbound sequence π_{in}:

$$r_{\pi_{in,1}} \geq r_{\pi_{in,2}} \geq \cdots \geq r_{\pi_{in,|S_{in}|}}$$

The Chromosome Assembly

Concatenate the base station S_0, the outbound sequence, the inbound sequence, and the return to the base station to generate the final initial tour π:

$$\pi = (S_0, \pi_{out}, \pi_{in}, S_0)$$

The Mathematical Guarantee

Because the radial distance strictly increases during π_{out} and strictly decreases during π_{in}, the sequence reaches a maximum distance exactly once. This mathematically guarantees the structural constraint for every individual in the initial population:

$$m(\pi) = 1$$

0.9 Curvature-aware geodesic partitioning

Traditional network partitioning algorithms (like NPA) use Euclidean (straight-line) distances and split regions by their geometric centroid. This fails when concave polygonal obstacles are present, because two sensor nodes can be geographically adjacent but metrically distant due to a dividing wall. We propose a Curvature-aware geodesic partitioning to replace Euclidean geometry with geodesic (visibility-based) geometry. It aims to design a geodesic bisection based on the ratio of the vehicle's minimum turning radius (R) to the mean nearest-neighbour spacing of the sensors.

The steps of which is as follows:

0.9.1 Geodesic Spectral Partitioning

The sensor partitioning process consists of the following steps:

1. **Geodesic Distance Matrix:**

   First, construct the geodesic distance matrix

   $$D_{\text{geo}} = [d_{\text{geo}}(i,j)]\,,$$

   where $d_{\text{geo}}(i,j)$ denotes the shortest obstacle-respecting distance between sensors S_i and S_j.

2. **Affinity Matrix:**

   Using the geodesic distances, construct the affinity matrix W as

   $$W_{ij} = \exp\left(-\frac{d_{\text{geo}}(i,j)^2}{\sigma^2}\right).$$

   For spectral partitioning, the diagonal elements are set to zero:

   $$W_{ii} = 0.$$

3. **Degree Matrix:**

   Calculate the degree of each sensor as

   $$d_i = \sum_j W_{ij}.$$

   The corresponding degree matrix is

   $$D = \begin{bmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{bmatrix}.$$

4. **Normalized Laplacian:**

   The normalized Laplacian matrix is then calculated as

$$L_{\text{norm}} = I - D^{-1/2} W D^{-1/2}.$$

5. **Eigenvalue and Eigenvector Computation:**

   The eigenvalues and eigenvectors of L_{norm} are obtained by solving

$$\det\left(L_{\text{norm}} - \lambda I\right) = 0.$$

   The eigenvalues are sorted in ascending order as

$$\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n.$$

6. **Fiedler Vector:**

   The Fiedler vector is the eigenvector corresponding to the second-smallest eigenvalue λ_2:

$$\boxed{\text{Fiedler vector} = v_2}.$$

7. **Sensor Partitioning:**

   The entries of the Fiedler vector are used to generate candidate sensor partitions by applying different threshold values. For a threshold t, the two partitions are defined as

$$A_t = \{S_i : f_i \leq t\},$$

$$B_t = \{S_i : f_i > t\},$$

   where f_i is the Fiedler-vector value corresponding to sensor S_i.

   Each candidate partition is evaluated using the estimated workload criterion

$$\min_t \max\left(\hat{T}(A_t), \hat{T}(B_t)\right).$$

   The threshold producing the best partition is selected.

0.10 Corridor Games

CorridorGame is proposed for resolving congestion when multiple mobile sinks (MSs) need to use the same narrow corridor. The CorridorGame is executed after generating the routes of all mobile sinks. The following steps are performed:

1. **Identify narrow corridors:** For each corridor $c \in C$, check whether

$$w_c < 2(R + \text{clearance}).$$

2. **Identify corridor users:** For every narrow corridor c, determine the set of mobile sinks whose routes traverse the corridor:

$$\text{users}(c) = \{k \mid \Gamma_k \text{ traverses } c\}.$$

3. **Detect congestion:** Compare the number of mobile sinks using the corridor with its capacity. If

$$|\text{users}(c)| > \text{capacity}(c),$$

   the corridor is considered congested and the game is activated.

4. **Define the players and strategies:** The mobile sinks using the congested corridor are the players. Each player is given three possible strategies:

$$\mathcal{S}_c = \{\text{Left}, \text{Right}, \text{Detour}\}.$$

5. **Game-theory step:** Each mobile sink evaluates the cost/latency associated with each feasible strategy and selects the strategy that gives the minimum cost.

6. **Check mission feasibility:** If a selected strategy causes the mission time of a mobile sink to exceed the allowed budget,

$$T_k > B,$$

the strategy is rejected and the previous route is retained.

7. **Best-response update:** The mobile sinks update their strategies one at a time according to their best response. This process is repeated until no mobile sink can improve its own cost by changing its strategy.

8. **Return updated routes:** After the process converges, the resulting set of mobile-sink routes is returned as

$$\Gamma'.$$

Algorithm 1: CorridorGame

Input: Existing mobile-sink routes Γ and set of corridors C
Output: Updated mobile-sink routes Γ'
foreach *corridor* $c \in C$ **do**
  Check whether

$$w_c < 2(R + \text{clearance})$$

  holds;
  Identify the set of mobile sinks using corridor c:

$$\text{users}(c) = \{k \mid \Gamma_k \text{ traverses } c\}$$

  ;
  if $|\text{users}(c)| > \text{capacity}(c)$ **then**
    Game-Theory Step:;
    Define the players as the mobile sinks using corridor c.;
    Give each player three possible strategies:

$$\mathcal{S}_c = \{\text{Left}, \text{Right}, \text{Detour}\}$$

    ;
    Each mobile sink evaluates the cost/latency of each feasible strategy and chooses the strategy with the minimum cost.;
    Repeat the best-response updates until no mobile sink can improve its own cost by changing its strategy.;
    if *the new route of any mobile sink violates* $T_k > B$ **then**
      Reject the change and restore its previous route;
return Γ';

0.10.1 Conflict Game in the Code

This is the theory of `conflict_game.py`. The code plays the game of Section 0.10 on whole timed trajectories, so corridors are not extracted (steps 1 to 3): a game starts wherever two sinks come too close. The strategies are GO, WAIT and DETOUR in place of Left, Right and Detour. Numbers are the defaults in `config.py`.

1. **Conflict detection:** Positions $p_k[n] = \Gamma_k(n\Delta t)$ are sampled every $\Delta t = 0.5$ s. Between samples the sinks move in straight lines. With $r_{kl} = p_k - p_l$, the closest distance in interval n is

$$\boxed{d_{kl}[n] = \min_{0 \le \tau \le 1} \| \, r_{kl}[n] + \tau\,(r_{kl}[n+1] - r_{kl}[n]) \, \|.}$$

   The interval is unsafe, $u_{kl}[n] = 1$, if $d_{kl}[n] < d_{\text{safe}}$, unless both sinks are within 5 m of the base at that closest moment. It is checked only while both sinks are on their mission (eq. 20). The exposure from frame s on is

$$E_{kl} = \Delta t \sum_{n \ge s} u_{kl}[n].$$

2. **When a game starts:** Let f be the earliest unsafe interval. The sinks unsafe in interval f are the players N. They decide 5 frames (2.5 s) earlier, at $s = \max(1, f - 5)$. All other sinks keep their trajectories. The search then continues from $f + 1$, so conflicts are solved one game at a time, in time order (at most 500 games).

3. **Strategies:** Each player has a set $A_k \subseteq \{\text{GO}, \text{WAIT}, \text{DETOUR}\}$, built once before the game.
   - GO: continue along the current trajectory. This is always available.
   - WAIT: stop at frame s for w_k frames, then continue. w_k is the shortest stop with the lowest risk against all other sinks, and $w_k \Delta t \le \min(20\,\text{s},\ B - T_k)$.
   - DETOUR: side-step $h = d_{\text{safe}} + 0.5 = 3.5$ m to the right (left only if right is infeasible) while the conflict lasts, then rejoin the own path. Offered only if the sink is moving, the ramps are at least $\pi\sqrt{hR/2}$ long, the curvature stays within $1.15/R$, and the path keeps r_{safe} from all obstacles.

   A strategy with $T_k + \Delta T_k > B$ is removed (Section 0.10, step 6).

4. **Cost function:** Each player minimises its own cost:

$$\boxed{J_k = \alpha\,\rho_k + \beta\,\Delta T_k + \gamma\,\frac{\Delta D_k}{v} + \delta\,W_k, \qquad \rho_k = \sum_{j \in N,\ j \ne k} E_{kj} + F_k.}$$

   ρ_k is the collision risk with the other players and with the non-players (F_k). ΔT_k, ΔD_k and W_k are the extra time, extra distance and waiting time. All terms are in seconds, and $\beta = \gamma = \delta = 1$. With $c_k = \beta\Delta T_k + \gamma\Delta D_k / v + \delta W_k$, the collision weight of each game is

$$\boxed{\alpha = \frac{1}{\Delta t}\Big(1 + \sum_{k \in N} \big\lfloor \max_{A_k} c_k - \min_{A_k} c_k \big\rceil\Big),}$$

   so one interval of risk costs more than any possible saving: no player accepts more risk to save time.

5. **Best response:** Everyone starts on GO. In each round every player computes its best response and its gain:

$$s_k^* = \arg\min_{s_k \in A_k} J_k(s_k, s_{-k}).$$

   Only the player with the largest gain switches (ties: shorter tour, then lower index). The process stops when no player can lower its cost by more than 10^{-9} by changing alone. This is a **pure Nash equilibrium**.

6. **Why it always stops:** The game has an exact potential function:

$$\boxed{\Phi = \alpha\Big(\sum_{k<j} E_{kj} + \sum_k F_k\Big) + \sum_k c_k, \qquad \Delta\Phi = \Delta J_k \quad \text{(only player } k \text{ changes).}}$$

   This holds because $E_{kj} = E_{jk}$, so the danger is charged equally to both sinks. Each strategy changes

only its own sink and is fixed before the game, non-players do not change, and the weights stay constant within a game. Every switch therefore lowers Φ, no combination is visited twice, and best response stops after at most $\Pi_k|A_k| - 1$ moves.

Important limitation: the equilibrium is stable, but not necessarily conflict-free, and not necessarily the combination with the lowest Φ. E_{kl} counts risk to the end of the mission, so a stop is also charged for later conflicts that a later game could still solve. Remaining conflicts are reported by the code, not hidden.

0.10.2 Simple Resolver: Wait Your Turn

This is the theory of `simple_resolver.py`, which can replace the game in Steps 9 and 10 of Section 0.11. It has no players, costs or equilibrium. Numbers are the defaults in `config.py`.

1. **Conflict detection:** Each step from frame n to $n+1$ is looked at five times:

$$\boxed{p_k(n,t) = p_k[n] + t\,(p_k[n+1] - p_k[n]), \qquad t \in \{0,\ 0.25,\ 0.5,\ 0.75,\ 1\}.}$$

   Step n is a conflict, $u_{kl}[n] = 1$, if $\|p_k(n,t) - p_l(n,t)\| < d_{\text{safe}}$ at any look, unless both sinks are within 5 m of the base at that look. It is checked only while both sinks are on their mission (eq. 20).

2. **Order of turns:** Sinks take turns by decreasing mission time (ties: lower index first):

$$\boxed{T_{(1)} \geq T_{(2)} \geq \cdots \geq T_{(K)}.}$$

   The longest tour has the least spare time, so it goes first and never waits. Each sink k looks only at D_k, the sinks that have already had their turn. Their trajectories are final, so every pair is handled once, by the sink later in the order.

3. **First meeting:** Starting from $n_0 = 1$, sink k finds its first conflict with a sink in D_k. If there is none, its turn ends.

$$\boxed{f = \min\ \{\, n \geq n_0 \ :\ u_{kj}[n] = 1 \text{ for some } j \in D_k \,\}.}$$

4. **The stop:** The sink stops at frame $s = \max(1, f-5)$, which is 2.5 s before the meeting, for w frames. Its new trajectory is

$$\boxed{p'[n] = p[n] \ \ (n \leq s), \qquad p'[n] = p[s] \ \ (s < n \leq s+w), \qquad p'[n] = p[n-w] \ \ (n > s+w).}$$

   This adds $w\Delta t$ to T_k and nothing to the path length. The stop is limited by 20 s and by the time left in the budget, $w_{\max} = \min(40,\ \lfloor (B - T_k)/\Delta t \rfloor)$. The sink takes the shortest stop after which the way is clear, from the stop until 5 s (10 frames) after it reaches the meeting point:

$$\boxed{w^* = \min\ \{\, w \leq w_{\max} \ :\ u'_{kj}[n] = 0 \text{ for all } j \in D_k,\ \ s \leq n < f + w + 10 \,\}.}$$

   Here u' is the conflict test with the new trajectory. If w^* exists, the stop is applied and the search continues from $n_0 = f + w^*$. If not, this conflict is left in place and the search continues from $n_0 = f + 1$.

5. **Why it always stops:** Each sink has exactly one turn. Within a turn n_0 grows after every meeting, and the trajectory can grow by at most $\lfloor (B - T_k)/\Delta t \rfloor$ frames in total.

6. **What holds for every input:**
   - Path unchanged: a stop only repeats positions, so the path length, obstacle clearance (eq. 19) and turning radius stay as planned.
   - Budget (eq. 18): a stop never pushes T_k over B.
   - Priority: the longest tour never changes, and a sink never changes after its turn.

Important limitation: the result is not guaranteed to be conflict-free or to have the least waiting. Waiting cannot solve a head-on meeting on one line, because the other sink drives through the stopped one, and a sink with no time left in its budget cannot wait. The order is fixed and never optimised. Away from the base, five looks per step can miss only a pass whose closest distance is between 2.93 m and 3 m. Conflicts that remain are reported by the separate closest-distance check in `planner.py`, not hidden.

0.11 Overall algorithm

Step 1: Initialize with $k = 1$ MS and assign all sensors to it.

Step 2: Construct the curvature-constrained feasible graph and compute Dubins/geodesic travel costs.

Step 3: Generate a radially unimodal ordering and optimize it using GA to obtain π^*.

Step 4: Compute $T_{\min} = T(\pi^*)$.

Step 5: $\begin{cases} T_{\min} \le B: & \text{Accept the solution and terminate.} \\ T_{\min} > B: & \text{Perform curvature-aware geodesic partitioning.} \end{cases}$

Step 6: For every partition P_i, independently optimize its tour and compute $T_i^{\min}$.

Step 7: $\begin{cases} \exists i : T_i^{\min} > B: & \text{Further partition } P_i \text{ and repeat Step 6.} \\ T_i^{\min} \le B,\ \forall i: & \text{Proceed to conflict detection.} \end{cases}$

Step 8: Detect conflicts among the mobile-sink trajectories.

Step 9: $\begin{cases} \text{No conflict:} & \text{Accept the partitioning and routes.} \\ \text{Conflict exists:} & \text{Apply CorridorGames to the conflicting MSs.} \end{cases}$

Step 10: Update the affected trajectories/schedules according to the CorridorGames conflict-resolution mechanism.

Step 11: Verify that the resulting routes satisfy Dubins feasibility and the time constraint $T_i \le B$.

Step 12: If all constraints are satisfied, accept the final solution; otherwise, re-optimize the affected routes and repeat.

==========

0.12 Background

A Wireless Sensor Network(WSN) comprises a large number of low-cost, low-power, small sized and multi-functional sensor nodes S deployed in a region of interest. These sensors are equipped with microprocessors and radio transceivers having data processing and communicating abilities. These sensors are typically battery powered and cannot be used for a long period of time. In most cases it is difficult to recharge them[1]. The individual senser nodes in a WSN are inherently resource constrained: they have limited processing speed, storage capacity, and communication bandwidth. The WSN contains a A sink or base station BS which acts like an interface between users and the network. One can retrieve required information from the network by injecting queries and gathering results from the sink. [2]Thus each sensor node must periodically send all the gathered information to the Base Station(BS) due to its resource constraints, such as finite storage or application requiring real time data.

The data can be forwarded to the BS through multi-hop transmission using cluster-based or hierarchical routing techniques. However, the sensors near the BS may exhaust their energy faster than other sensors because of their frequent data forwarding and receiving activities, which may lead to energy hole around the BS. However, the sensors near the BS may exhaust their energy faster than other sensors because of their frequent data forwarding and receiving activities, which may lead to energy hole around the BS One promising solution is the adoption of Mobile Sinks (SNKs),which travel around the monitoring area to collect data from the sensors. Using an SNK reduces sensors' energy consumption in data receiving and forwarding, thereby extending the lifetime of WSNs [3].

A mobile sink is able to follow a real-life movement pattern such as pedestrian mobility pattern, vehicular mobility pattern and dynamic mobility pattern. In the pedestrian mobility pattern, the movement follows

the characteristics of walking people that are equipped with sensor nodes attached to their body. In vehicular mobility pattern, the mobility of the mobile sink depends on the characteristics of the vehicle on which it is mounted. In the dynamic mobility pattern, the movement is based on the medium that the node moves through, such as wind or water.[4]

In order to plan the autonomous movement of a mobile sink, both path planning and motion planning needs to be considered. Path planning determines where the mobile sink should go. Motion planning involves how and where the mobile sink should move while considering the vehicle kinematics and dynamics. Motion planning is one of the sub task of task assignment apart from cooperation, coordination with other mobile sinks and collision avoidance with the obstacles.

This paper addresses the problem of task assignment and motion planning of a non- holonomic vehicle or more specifically a Dubins vehicle. A Dubins vehicle is one that moves only forward at a fixed speed and has a minimum radius of turn constraint[5]. It was proved that such an optimal Dubins path can have at most 3 segments (where segments can have zero length) that can be categorized in two main types: • CCC type: LRL, RLR; • CSC type: LSL, LSR, RSL, RSR. Its kinematics are given by:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = v \begin{bmatrix} \cos\theta \\ \sin\theta \\ \frac{\alpha}{R} \end{bmatrix}, \qquad \alpha \in [-1, 1] \tag{21}$$

where $\alpha \in [-1, 1]$ is the normalized curvature control input, with $\alpha = -1$, $\alpha = 0$, and $\alpha = 1$ corresponding to maximum right turn, straight motion, and maximum left turn, respectively.

0.13 Mathematical Preliminaries

Proposed changes to Section 0.13. Blue lines are additions. Lines shaded red should be removed; the red note below them gives the reason. No original line has been removed.

Given two initial positions

$$(x_1, y_1, \psi_1), \qquad (x_2, y_2, \psi_2),$$

we are to find the shortest Dubins path of minimum turn radius R.

Note. The rule below picks one turn type from the signs of cp_1 and cp_2. It does not always give the shortest path. The true shortest path is found by computing S_{path} for all six turn types (LSL, RSR, LSR, RSL, RLR, LRL) with the formulas below and keeping the smallest.

Define

$$\hat{u}_1 = \begin{bmatrix} \cos\psi_1 \\ \sin\psi_1 \end{bmatrix},$$

$$\hat{u}_2 = \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \end{bmatrix},$$

$$\hat{u}_3 = \begin{bmatrix} \cos\psi_2 \\ \sin\psi_2 \end{bmatrix}.$$

Finding the cross product of the above two vectors determines the direction:

$$cp_1 = \hat{u}_1 \times \hat{u}_2,$$

$$cp_2 = \hat{u}_2 \times \hat{u}_3.$$

Only the signs of cp_1 and cp_2 are used, so $\hat{u}_2$ need not have unit length. This also avoids dividing by zero when the two points coincide.

The turn type is determined as follows:

$$\begin{cases} cp_1 \le 0,\ cp_2 > 0 & \Rightarrow \text{turn type} = RSL, \qquad \sigma_1 = -1,\ \sigma_2 = 1, \\ cp_1 \le 0,\ cp_2 < 0 & \Rightarrow \text{turn type} = RSR, \qquad \sigma_1 = -1,\ \sigma_2 = -1, \\ cp_1 > 0,\ cp_2 < 0 & \Rightarrow \text{turn type} = LSR, \qquad \sigma_1 = 1,\ \sigma_2 = -1, \\ cp_1 > 0,\ cp_2 > 0 & \Rightarrow \text{turn type} = LSL, \qquad \sigma_1 = 1,\ \sigma_2 = 1. \end{cases}$$

The table above does not cover $cp_2 = 0$, where the end heading lies along the line joining the two points. This case is treated like $cp_2 > 0$.

Then we find the circle centers:

$$(x_{ci}, y_{ci}) = \left(x_i + R\cos\left(\psi_i + \frac{\pi}{2}\sigma_i\right),\ y_i + R\sin\left(\psi_i + \frac{\pi}{2}\sigma_i\right)\right).$$

Finding the distance between the two circle centers:

$$S_{\tan} = \sqrt{(x_{c2} - x_{c1})^2 + (y_{c2} - y_{c1})^2}.$$

If

$$2R \le S_{\tan} < 3R,$$

then it is a CCC curve. Else it is a CSC curve

R1. Remove the three shaded lines. They are not true for LSR and RSL, which stay CSC curves when $2R \le S_{\tan} < 3R$. The two cases below already say when a CCC curve is used.

Only RSR and LSL can become CCC curves, as the two cases below state. Every other case is a CSC curve.

If the turn type is RSR and $2R \le S_{\tan} < 3R$, then

$$\text{turn type} = RLR, \qquad \sigma_1 = \sigma_2 = -1.$$

If the turn type is LSL and $2R \le S_{\tan} < 3R$, then

$$\text{turn type} = LRL, \qquad \sigma_1 = \sigma_2 = 1.$$

Existence of each turn type. RSR and LSL exist for every $S_{\tan}$. LSR and RSL exist only if $S_{\tan} \ge 2R$: when the two circles are closer than that they overlap, and the straight line these paths need does not exist. RLR and LRL exist only if $S_{\tan} \le 4R$, because the middle circle must touch both.

If the rule picks LSR or RSL with $S_{\tan} < 2R$, that path does not exist. The shortest of the turn types that do exist is used instead.

A CCC curve can exist up to $S_{\tan} = 4R$ and can be the shortest path above $3R$, but the rule tries it only for $2R \le S_{\tan} < 3R$.

Finding trajectory of CSC curve:

$$\psi_L = \frac{y_{c2} - y_{c1}}{x_{c2} - x_{c1}}.$$

R2. Remove the shaded line. ψ_L is an angle, but this ratio is only its tangent, and it divides by zero when $x_{c2} = x_{c1}$.

$$\psi_L = \operatorname{atan2}(y_{c2} - y_{c1},\ x_{c2} - x_{c1}).$$

If the turn type is RSR or LSL, then

$$\text{cut-out angle } \psi_d = \psi_L.$$

If the turn type is LSR, then

$$\text{cut-out angle } \psi_d = \psi_L + \tan^{-1}\left(\frac{2R}{S_{\tan}}\right).$$

R3. Remove the shaded line. $\tan^{-1}$ would be right only if $S_{\tan}$ were the length of the straight part. $S_{\tan}$ is the distance between the circle centres: the hypotenuse of a right triangle whose other two sides are $2R$ and the straight part. At $S_{\tan} = 3R$ the $\tan^{-1}$ angle is wrong by 8.1 degrees.

$$\text{cut-out angle } \psi_d = \psi_L + \sin^{-1}\left(\frac{2R}{S_{\tan}}\right).$$

If the turn type is RSL, then

$$\text{cut-out angle } \psi_d = \psi_L - \tan^{-1}\left(\frac{2R}{S_{\tan}}\right).$$

R4. Remove the shaded line, for the same reason as R3.

$$\text{cut-out angle } \psi_d = \psi_L - \sin^{-1}\left(\frac{2R}{S_{\tan}}\right).$$

The length of the straight part is $d_{co,\,ci} = S_{\tan}$ for RSR and LSL, and $d_{co,\,ci} = \sqrt{S_{\tan}^2 - 4R^2}$ for LSR and RSL. Below, ψ_D is the same angle as ψ_d.

0.13.1 CSC Trajectory

Let the angular increment be

$$\Delta\psi = 0.05.$$

R5. Remove the two shaded lines. The exact arcs added below do not need an angular increment.

Each arc length follows directly from the headings, taking each angle in the range 0 to 2π:

$$S_1 = R\,[\sigma_1(\psi_D - \psi_1) \bmod 2\pi], \qquad S_2 = R\,[\sigma_2(\psi_2 - \psi_D) \bmod 2\pi].$$

Let $\alpha_1 = S_1/R$ and $\alpha_2 = S_2/R$ be the two turning angles.

First circular arc: Cut-out point The cut-out point from the first circular arc is given by

$$x_{co} = \cos\alpha_1\,(x_1 - x_{c1}) - \sigma_1 \sin\alpha_1\,(y_1 - y_{c1}) + x_{c1},$$
$$y_{co} = \sigma_1 \sin\alpha_1\,(x_1 - x_{c1}) + \cos\alpha_1\,(y_1 - y_{c1}) + y_{c1}.$$

$$\begin{bmatrix} x_{co} \\ y_{co} \end{bmatrix} = \begin{bmatrix} \cos(i\Delta\psi) & -\sigma_1 \sin(i\Delta\psi) \\ \sigma_1 \sin(i\Delta\psi) & \cos(i\Delta\psi) \end{bmatrix} \begin{bmatrix} x_1 - x_{c1} \\ y_1 - y_{c1} \end{bmatrix} + \begin{bmatrix} x_{c1} \\ y_{c1} \end{bmatrix}.$$

Initially,

$$x_{t1} = x_1, \qquad y_{t1} = y_1, \qquad x_{co} = x_1, \qquad y_{co} = y_1,$$
$$\psi_T = \psi_1, \qquad i = 1.$$

The cut-out point is obtained by iterating until

$$\boxed{|\psi_T - \psi_D| \le 0.1}$$

where the heading is updated as

$$\psi_T = \psi_T + \sigma_1 \Delta\psi.$$

The arc-length increment at each iteration is

$$\Delta S = R\Delta\psi = 0.05R.$$

Hence, the accumulated distance along the first circular arc is

$$S_1 = \sum_{i=1}^{n_1} R\Delta\psi = n_1 R\Delta\psi.$$

R6. Remove all the shaded lines of this first arc. Stepping stops when the heading is within 0.1 rad of ψ_D, so the heading can be wrong where two parts meet, the parts can meet at a small corner, and S_1 is only approximate. If the heading starts just past ψ_D, an almost full circle is skipped and the length is wrong by about $2\pi R$. The exact formulas added above replace these lines.

Second circular arc: Cut-in point Initially,

$$x_{t2} = x_2, \qquad y_{t2} = y_2, \qquad x_{ci} = x_2, \qquad y_{ci} = y_2,$$
$$\psi_T = \psi_2, \qquad i = 1.$$

The cut-in point from the second circular arc is given by

$$x_{ci} = \cos\alpha_2\,(x_2 - x_{c2}) + \sigma_2 \sin\alpha_2\,(y_2 - y_{c2}) + x_{c2},$$
$$y_{ci} = -\sigma_2 \sin\alpha_2\,(x_2 - x_{c2}) + \cos\alpha_2\,(y_2 - y_{c2}) + y_{c2}.$$

$$\begin{bmatrix} x_{ci} \\ y_{ci} \end{bmatrix} = \begin{bmatrix} \cos(i\Delta\psi) & \sigma_2 \sin(i\Delta\psi) \\ -\sigma_2 \sin(i\Delta\psi) & \cos(i\Delta\psi) \end{bmatrix} \begin{bmatrix} x_2 - x_{c2} \\ y_2 - y_{c2} \end{bmatrix} + \begin{bmatrix} x_{c2} \\ y_{c2} \end{bmatrix}.$$

The heading is updated according to

$$\psi_T = \psi_T - \sigma_2 \Delta\psi,$$

and the iteration continues until

$$|\psi_T - \psi_D| \le 0.1.$$

The accumulated distance along the second circular arc is

$$S_2 = \sum_{i=1}^{n_2} R\Delta\psi = n_2 R\Delta\psi.$$

R7. Remove all the shaded lines of this second arc, including the single word Initially in the heading line, for the same reason as R6. The exact formulas added above replace them.

Straight trajectory The overall trajectory consists of the first circular arc, the straight segment, and the second circular arc.

The direction vector from the cut-out point to the cut-in point is

$$\mathbf{u} = \begin{bmatrix} x_{ci} - x_{co} \\ y_{ci} - y_{co} \end{bmatrix}.$$

The corresponding unit vector is

$$\hat{\mathbf{u}} = \frac{\begin{bmatrix} x_{ci} - x_{co} \\ y_{ci} - y_{co} \end{bmatrix}}{\sqrt{(x_{ci} - x_{co})^2 + (y_{ci} - y_{co})^2}}.$$

The distance between the cut-out and cut-in points is

$$d_{co,ci} = \sqrt{(x_{ci} - x_{co})^2 + (y_{ci} - y_{co})^2}.$$

The straight trajectory is initialized with the trajectory generated by the first circular arc. The intermediate points are generated as

$$x_{\text{trajectory}}[j+1] = x_{\text{trajectory}}[j] + \Delta s\, \hat{u}_x,$$
$$y_{\text{trajectory}}[j+1] = y_{\text{trajectory}}[j] + \Delta s\, \hat{u}_y,$$

for $j = 1, \ldots, \lfloor d_{co,ci}/\Delta s \rfloor$, where Δs is the spacing between points. The same spacing is used on the arcs, that is, an angle of $\Delta s/R$ per point.

$$x_{\text{trajectory}}[j+1] = x_{\text{trajectory}}[j] + 3\hat{u}_x,$$
$$y_{\text{trajectory}}[j+1] = y_{\text{trajectory}}[j] + 3\hat{u}_y,$$

for

$$j = 1, \ldots, \left\lfloor \frac{d_{co,ci}}{3} \right\rfloor.$$

R8. Remove the shaded lines. They fix the spacing at 3 on the straight part only, so the points are unevenly spaced along the path. With one spacing Δs on every part, the time at each point follows from its distance along the path.

Finally, the cut-in point is appended to the trajectory:

$$\boxed{x_{\text{trajectory}} \leftarrow x_{\text{trajectory}} \cup \{x_{ci}\}}$$

$$\boxed{y_{\text{trajectory}} \leftarrow y_{\text{trajectory}} \cup \{y_{ci}\}.}$$

The total path distance is therefore

$$\boxed{S_{\text{path}} = S_1 + d_{co,ci} + S_2.}$$

The output trajectory is

$$\boxed{(x_{\text{trajectory}}, y_{\text{trajectory}}, S_{\text{path}})\,.}$$

0.13.2 CCC Trajectory

A CCC curve exists only if $S_{\tan} \le 4R$. Here $\sigma_1 = \sigma_2$, and the middle arc turns the other way. The first and middle circles touch at the cut-out point, and the middle and last circles touch at the cut-in point. Let

$$\theta = \cos^{-1}\left(\frac{S_{\tan}}{4R}\right).$$

The centre of the middle circle is

$$x_{c3} = x_{c1} + 2R\cos(\psi_L + \sigma_1\theta), \qquad y_{c3} = y_{c1} + 2R\sin(\psi_L + \sigma_1\theta).$$

The cut-out and cut-in points are

$$x_{co} = x_{c1} + R\cos(\psi_L + \sigma_1\theta), \qquad y_{co} = y_{c1} + R\sin(\psi_L + \sigma_1\theta),$$
$$x_{ci} = x_{c2} - R\cos(\psi_L - \sigma_1\theta), \qquad y_{ci} = y_{c2} - R\sin(\psi_L - \sigma_1\theta).$$

The headings at these two points are

$$\psi_{co} = \psi_L + \sigma_1\left(\theta + \frac{\pi}{2}\right), \qquad \psi_{ci} = \psi_L - \sigma_1\left(\theta + \frac{\pi}{2}\right).$$

The arc lengths, taking each angle in the range 0 to 2π, are

$$S_1 = R\,[\sigma_1(\psi_{co} - \psi_1) \bmod 2\pi], \qquad S_3 = R\,(\pi + 2\theta), \qquad S_2 = R\,[\sigma_2(\psi_2 - \psi_{ci}) \bmod 2\pi],$$

and the total path distance is

$$S_{\text{path}} = S_1 + S_3 + S_2.$$

0.14 Reference

[1] J. Zheng and A. Jamalipour, Eds., Wireless Sensor Networks: A Networking Perspective. Hoboken, NJ, USA: Wiley-IEEE Press, 2009.

[2] M. A. Matin and M. M. Islam, "Overview of Wireless Sensor Network," in Wireless Sensor Networks—Technology and Protocols, M. A. Matin, Ed. London, U.K.: InTech, 2012, pp. 1–22, doi: 10.5772/49376.

[3]R. Kumar and J. C. Mukherjee, "Approximation algorithms for vehicle-aided periodic data collection from mobile sensors with obstacle avoidance in WSNs," Ad Hoc Networks, vol. 149, Art. no. 103239, 2023, doi: 10.1016/j.adhoc.2023.103239.

[4]N. Temene, C. Sergiou, C. Georgiou, and V. Vassiliou, "A Survey on Mobility in Wireless Sensor Networks," Ad Hoc Networks, vol. 125, Article 102726, 2022. DOI: 10.1016/j.adhoc.2021.102726.

[5] Gottlieb, Y., Manathara, J. G., and Shima, T. (2018). "Multi-Target Motion Planning Amidst Obstacles for Autonomous Aerial and Ground Vehicles." Journal of Intelligent & Robotic Systems, 90(3–4), 515–536.